

使用 Modbus 协议进行通讯时，读数据命令功能码为 03H，写数据命令功能码为 10H。

起始地址 (16 进制)	起始地址 (10 进制)	数据项名称	长度 (字节)	读/ 写	备注	数据类型
0x0000	0	通信地址	2	R/W	1 ~ 247 (默认 1)	UInt16
0x0001	1	波特率	4	R/W	9600 38400 57600 115200 (默认值)	UInt32
0x0003	3	RTC 时间	4	R/W	UNIX 时间戳	UInt32
0x0005	5	温度采样周期 (秒)	2	R/W		UInt16
0x0006	6	合闸报警值	2	R/W	单位: 0.1ms(max 10000)	UInt16
0x0007	7	分闸报警值	2	R/W	单位: 0.1ms	UInt16
0x0008	8	合闸异常值	2	R/W	单位: 0.1ms	UInt16
0x0009	9	分闸异常值	2	R/W	单位: 0.1ms	UInt16
0x000A	10	温度异常值	2	R/W	Ex: 温度异常值为 50℃: 0x32	UInt16
0x000B	11	湿度异常值	2	R/W	0 ~ 120	UInt16
0x000C	12	温度差报警值	2	R/W	0 ~ 200	UInt16
0x000D	13	硬件版本号	2	R	0x01:一期 0x02:二期	UInt16
0x000E	14	内部温度值	2	R	Ex: 温度值为 32℃: 0x84 (132 - 100)	UInt16
0x000F	15	外部温度值	2	R	Ex: 温度值为 32℃: 0x84 (132 - 100)	UInt16
0x0010	16	湿度值	2	R	0 ~ 120	UInt16
0x0011	17	通道 4 报警	2	R	Bit0 ~ 7: 0x00:没有异常 0x01:有异常 Bit8 ~ 15: 0x00: channel 3 低电平 0x01: channel 3 高电平	UInt16
0x0012	18	是否有分闸信号	2	R	0x00: 不存在 0x01: 存在分闸信号	UInt16
0x0013	19	分闸时间	2	R	单位: 0.1ms(max 10000)	UInt16
0x0014	20	分闸发生时间	4	R	Unix time	UInt16
0x0016	22	设备唯一标识	8	R	IMEI: 国际移动设备识别码, 每个节点的唯一身份码, 15 位, 采用 16 位 BCD 码表	UInt16

					示，最高位填 0 Ex: IMEI: 869976030015284 - 0x0869976030015284	
0x001A	26	是否有合闸信号	2	R	0x00: 不存在 0x01: 存在合闸信号	Uint16
0x001B	27	合闸时间	2	R	单位: 0.1ms(max 10000)	Uint16
0x001C	28	合闸发生时间	4	R	Unix time	Uint16
0x001E	30	结露温度差值	2	R	X10 列如: 差值是 10.2 摄氏度, 该寄存器值为 102	Uint16
0x001F	31	结露温度值	2	R	(+100)X10 列如: 温度值是-10.2 摄氏度, 该寄存器值为 (-10.2+100)x10=898	Uint16
0x0020	32	温度最大值	2	R	(+100)X10	Uint16
0x0021	33	温度最小值	2	R	(+100)X10	Uint16
0x0022	34	温度最大最小差值	2	R	(+100)X10	Uint16
0x0023	35	本机状态	2	R	0: 正常 1: 异常	Uint16
0x0024~ 0x004F	36 ~ 79	reserved				
0x0050 ~ 0x013F	80 ~ 319	无线温度通道号	2	R		Uint16
0x0140 ~ 0x022F	320 ~ 559	无线温度值	2	R	-50~125.0℃ (×10)	int16
0x0230 ~ 0x031F	560 ~ 799	无线湿度通道号	2	R		Uint16
0x0320 ~ 0x040F	800 ~ 1039	无线湿度值	2	R		Uint16
0x0410	1040	本设备无线温度 1	2	W	-50~125.0℃ (×10)	Uint16
0x0411	1041	本设备无线温度 2	2	W	-50~125.0℃ (×10)	Uint16
0x0412	1042	本设备无线温度 3	2	W	-50~125.0℃ (×10)	Uint16
0x0413	1043	本设备无线温度 4	2	W	-50~125.0℃ (×10)	Uint16

0x0414	1044	本设备无线温度 5	2	W	-50~125.0℃ (×10)	Uint16
0x0415	1045	本设备无线温度 6	2	W	-50~125.0℃ (×10)	Uint16
0x0416	1046	本设备无线湿度	2	W	(×10)	Uint16
0x0417 ~ 0x042A	1047 ~ 1066	无线电量	2	R	单位：毫伏	Uint16
0x042B ~ 0x0452	1067 ~ 1106	绑定无线测温设备号	2	R/W	2 个寄存器一个设备，高位在前，解绑写入 0x0000	Uint16

一期：支持 0x0000 ~ 0x0016

二期：全部支持

读取分合闸时间逻辑：按一定时间轮询 0x0012，如果存在信号，则读取 0x0013, 0x0014，设备将认为该次时间已经读取，如果还有分合闸信号，重新读取 0x0012，然后读取 0x0013, 0x0014，以此类推。

写入只支持每次写入一个寄存器，针对特殊的 0x0001 和 0x0003 寄存器，写入时将寄存器个数为 2，除此之外都为 1。

读取时支持多个寄存器同时读取，单次最多读取 100 个寄存器。读取无线温度和湿度时，不要通道和数值放在一起读，单次只支持读取一项。

CRC 算法

```
static uint16_t crcTb[] = {
    0X0000, 0XC0C1, 0XC181, 0X0140, 0XC301, 0X03C0, 0X0280, 0XC241,
    0XC601, 0X06C0, 0X0780, 0XC741, 0X0500, 0XC5C1, 0XC481, 0X0440,
    0XCC01, 0X0CC0, 0X0D80, 0XCD41, 0X0F00, 0XCFC1, 0XCE81, 0X0E40,
    0X0A00, 0XCAC1, 0XCB81, 0X0B40, 0XC901, 0X09C0, 0X0880, 0XC841,
    0XD801, 0X18C0, 0X1980, 0XD941, 0X1B00, 0XDBC1, 0XDA81, 0X1A40,
    0X1E00, 0XDEC1, 0XDF81, 0X1F40, 0XDD01, 0X1DC0, 0X1C80, 0XDC41,
    0X1400, 0XD4C1, 0XD581, 0X1540, 0XD701, 0X17C0, 0X1680, 0XD641,
    0XD201, 0X12C0, 0X1380, 0XD341, 0X1100, 0XD1C1, 0XD081, 0X1040,
    0XF001, 0X30C0, 0X3180, 0XF141, 0X3300, 0XF3C1, 0XF281, 0X3240,
    0X3600, 0XF6C1, 0XF781, 0X3740, 0XF501, 0X35C0, 0X3480, 0XF441,
    0X3C00, 0XFCC1, 0XFD81, 0X3D40, 0XFF01, 0X3FC0, 0X3E80, 0XFE41,
```

```

    0XFA01, 0X3AC0, 0X3B80, 0XFB41, 0X3900, 0XF9C1, 0XF881, 0X3840,
    0X2800, 0XE8C1, 0XE981, 0X2940, 0XEB01, 0X2BC0, 0X2A80, 0XEA41,
    0XEE01, 0X2EC0, 0X2F80, 0XEF41, 0X2D00, 0XEDC1, 0XEC81, 0X2C40,
    0XE401, 0X24C0, 0X2580, 0XE541, 0X2700, 0XE7C1, 0XE681, 0X2640,
    0X2200, 0XE2C1, 0XE381, 0X2340, 0XE101, 0X21C0, 0X2080, 0XE041,
    0XA001, 0X60C0, 0X6180, 0XA141, 0X6300, 0XA3C1, 0XA281, 0X6240,
    0X6600, 0XA6C1, 0XA781, 0X6740, 0XA501, 0X65C0, 0X6480, 0XA441,
    0X6C00, 0XACC1, 0XAD81, 0X6D40, 0XAF01, 0X6FC0, 0X6E80, 0XAE41,
    0XAA01, 0X6AC0, 0X6B80, 0XAB41, 0X6900, 0XA9C1, 0XA881, 0X6840,
    0X7800, 0XB8C1, 0XB981, 0X7940, 0XBB01, 0X7BC0, 0X7A80, 0XBA41,
    0XBE01, 0X7EC0, 0X7F80, 0XBF41, 0X7D00, 0XBDC1, 0XBC81, 0X7C40,
    0XB401, 0X74C0, 0X7580, 0XB541, 0X7700, 0XB7C1, 0XB681, 0X7640,
    0X7200, 0XB2C1, 0XB381, 0X7340, 0XB101, 0X71C0, 0X7080, 0XB041,
    0X5000, 0X90C1, 0X9181, 0X5140, 0X9301, 0X53C0, 0X5280, 0X9241,
    0X9601, 0X56C0, 0X5780, 0X9741, 0X5500, 0X95C1, 0X9481, 0X5440,
    0X9C01, 0X5CC0, 0X5D80, 0X9D41, 0X5F00, 0X9FC1, 0X9E81, 0X5E40,
    0X5A00, 0X9AC1, 0X9B81, 0X5B40, 0X9901, 0X59C0, 0X5880, 0X9841,
    0X8801, 0X48C0, 0X4980, 0X8941, 0X4B00, 0X8BC1, 0X8A81, 0X4A40,
    0X4E00, 0X8EC1, 0X8F81, 0X4F40, 0X8D01, 0X4DC0, 0X4C80, 0X8C41,
    0X4400, 0X84C1, 0X8581, 0X4540, 0X8701, 0X47C0, 0X4680, 0X8641,
    0X8201, 0X42C0, 0X4380, 0X8341, 0X4100, 0X81C1, 0X8081, 0X4040
};

uint16_t Modbus_Crc_Compute(const uint8_t *buf, uint16_t bufLen) {
    uint8_t num;
    uint16_t modbus16 = UINT16_MAX;
    uint16_t index = 0;
    for (index = 0; index < bufLen; ++index) {
        num = (uint8_t) (modbus16 & UINT32_MAX);
        modbus16 = (uint16_t) (((uint32_t) modbus16 >> 8) ^ crcTb[(num ^ buf[index]) &
UINT8_MAX]);
    }
    return modbus16;
}

```